MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Daniel Moisei FAF-241

Report

Laboratory Work No. 4

Operating Systems

Topic: The xv6 Shell from the Outside

Checked by:
Reițman Patricia, *university assistant*

Chișinău - 2026

CONTENTS

Introduction **3**

1 Fix sleep for the Current xv6 **4**

1.1 Identify the system-call name 4

1.2 Updated program and Makefile entry 4

1.3 Build and test 5

2 Bash versus the xv6 Shell **6**

2.1 Comparison table 6

2.2 Observations 8

3 Who Expands the Argument Vector **10**

3.1 Argument-printing programs 10

3.2 Observed argument behavior 10

4 Read the Shell Source **13**

4.1 Command representations and dispatch 13

4.2 Why cd is special 14

4.3 Connection to the experiments 15

Conclusion **16**

References **17**

INTRODUCTION

The purpose of this laboratory work was to study the xv6 shell as a user-visible program and compare it with Bash. The work verified that the `sleep` utility uses the current xv6 system-call name, compared shell syntax and interactive conveniences, used a small argument-printing program to reveal exactly what a process receives in `argv`, and connected the observed behavior to the implementation in `user/sh.c`.

The current local xv6 source tree declares `int pause(int);` in `user/user.h`. Its shell source contains 499 lines and defines five internal command representations. The report combines those source observations with captured Bash and xv6 sessions, allowing the written explanations to be checked directly against the displayed commands and output.

1 FIX SLEEP FOR THE CURRENT XV6

1.1 Identify the system-call name

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

The current tree printed `25:int pause(int);`. Therefore the user program must call `pause`, not the older `sleep` system-call wrapper. The command name may still be `sleep`; only the C interface used inside the program changed.



Figure 1.1: The pause declaration found in user/user.h

1.2 Updated program and Makefile entry

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1]));
    exit(0);
}
```

The source validates that exactly one argument was provided, converts the tick count from text to an integer, pauses the process, and exits successfully. The Makefile must include a tab-indented `$U/_sleep\` entry in `UPROGS`; the trailing backslash must be the final character on its line.

```
❯ nano user/sleep.c
❯ nano Makefile
❯ nl -ba user/sleep.c
grep -n '\$U/_sleep' Makefile
     1  #include "kernel/types.h"
     2  #include "user/user.h"
     3
     4  int
     5  main(int argc, char *argv[])
     6  {
     7      if(argc != 2){
     8          fprintf(2, "usage: sleep <ticks>\n");
     9          exit(1);
    10      }
    11
    12      pause(atoi(argv[1]));
    13      exit(0);
    14  }
153:    $U/_sleep\
```

Figure 1.2: Updated sleep.c and its UPROGS entry

1.3 Build and test

```
make qemu
$ sleep 20
$ sleep
usage: sleep <ticks>
```

The first invocation blocks for approximately 20 timer ticks and then returns to the prompt. The second invocation exercises argument validation and prints the usage diagnostic because no tick count was supplied.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sleep 20
sleep
$ usage: sleep <ticks>
$
```

Figure 1.3: The sleep program running with and without a tick argument

2 BASH VERSUS THE XV6 SHELL

2.1 Comparison table

The Linux test directory contained `README`; the first redirection test also created `f`. Table 1 summarizes the expected output and the reason for each difference. Exact spacing and process identifiers are environment-dependent.

Table 1: Bash and xv6 shell behavior

Command	**Feature**	**Bash**	**xv6 shell**
`echo hello > f ; cat < f`	redirection	Prints `hello`.	Prints `hello`; both input and output redirection are implemented.
`ls | grep READ`	pipe	Prints the matching `README` entry.	Prints the matching `README` entry.
`(echo a; echo b) | wc`	grouping and list	Reports 2 lines, 2 words, and 4 bytes.	Produces the same counts; parentheses group a parsed command tree.
`echo one ; echo two`	sequence	Prints `one`, then `two`.	Prints `one`, then `two`.
`echo bg &`	background	Runs asynchronously and usually prints a job number and PID.	Runs asynchronously, but there is no Bash-style job table or job-control message.
`ls *`	globbing	Bash expands * to matching names before executing `ls`.	The literal name * is passed to `ls`, which cannot open it.
`echo $HOME`	variables	Prints the value of the home-directory variable.	Prints the literal text `$HOME`.
`echo "a b"`	quotes	Prints `a b` as one argument, without quote characters.	Quotes have no special meaning; they remain in the tokens and the embedded space still separates arguments.
`mkdir d ; cd d ; ls ..`	path and built-in	Changes to `d`, then finds `ls` through `PATH` and lists the parent directory.	Prints `exec cd failed`; the shell stays in the root directory, so `ls ..` lists that directory. After a standalone `cd d`, plain `ls` fails because xv6 performs no path search.
Arrow Up, Tab, Ctrl+R	history and completion	Recalls history, completes input, and searches history.	No command history, completion, or reverse search is implemented.

```
moiseid@fedora:~/xv6-riscv$ mkdir -p ~/shtest
cd ~/shtest
touch README
moiseid@fedora:~/shtest$ echo hello > f ; cat < f
hello
moiseid@fedora:~/shtest$ ls | grep READ
README
moiseid@fedora:~/shtest$ (echo a; echo b) | wc
      2       2       4
moiseid@fedora:~/shtest$ echo one ; echo two
one
two
moiseid@fedora:~/shtest$ echo bg &
[1] 84117
bg
[1]+  Done                    echo bg
moiseid@fedora:~/shtest$ ls *
f  README
moiseid@fedora:~/shtest$ echo $HOME
/home/moiseid
moiseid@fedora:~/shtest$ echo "a b"
a b
moiseid@fedora:~/shtest$ mkdir -p d ; cd d ; ls ..
d  f  README
moiseid@fedora:~/shtest/d$ cd ..
moiseid@fedora:~/shtest$
```

Figure 2.1: The comparison commands executed in Bash

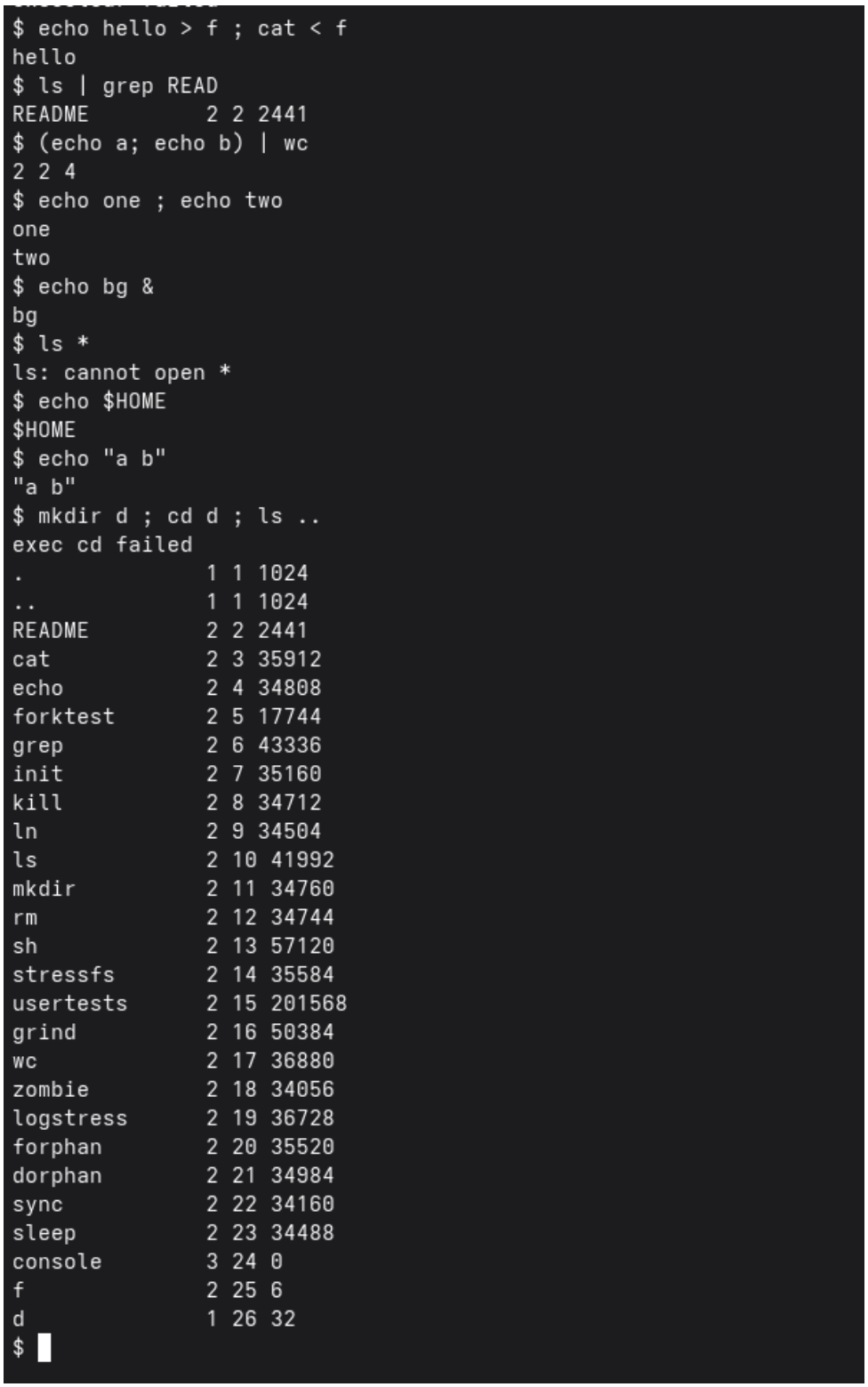


Figure 2.2: The comparison commands executed in the xv6 shell

2.2 Observations

Both shells support executing programs, input and output redirection, pipes, semicolon-separated lists, background execution, and parenthesized grouping. Bash additionally performs variable expansion, pathname expansion, quote removal, command lookup through `PATH`, command history, completion, reverse history search, and interactive job control. In the captured compound xv6 command, `mkdir d ; cd d ; ls ..`, the shell printed `exec cd failed`: the embedded `cd` was parsed as an external command, so the shell

stayed in the root directory and the following `ls ..` listed that directory.

After the xv6 shell processes a standalone `cd d`, its current directory is `d`. The shell passes `ls` directly to `exec`; it does not search a list of program directories. Because the xv6 user programs are stored in the root directory, the working alternative is:

```
$ ../ls ..
```

The relative path `../ls` identifies the executable, and the second `..` is the directory argument to list. Figure 2.3 confirms both stages: a standalone `cd d` changes the shell's directory, plain `ls ..` then fails because xv6 performs no path search, and `../ls ..` succeeds by naming the executable explicitly.

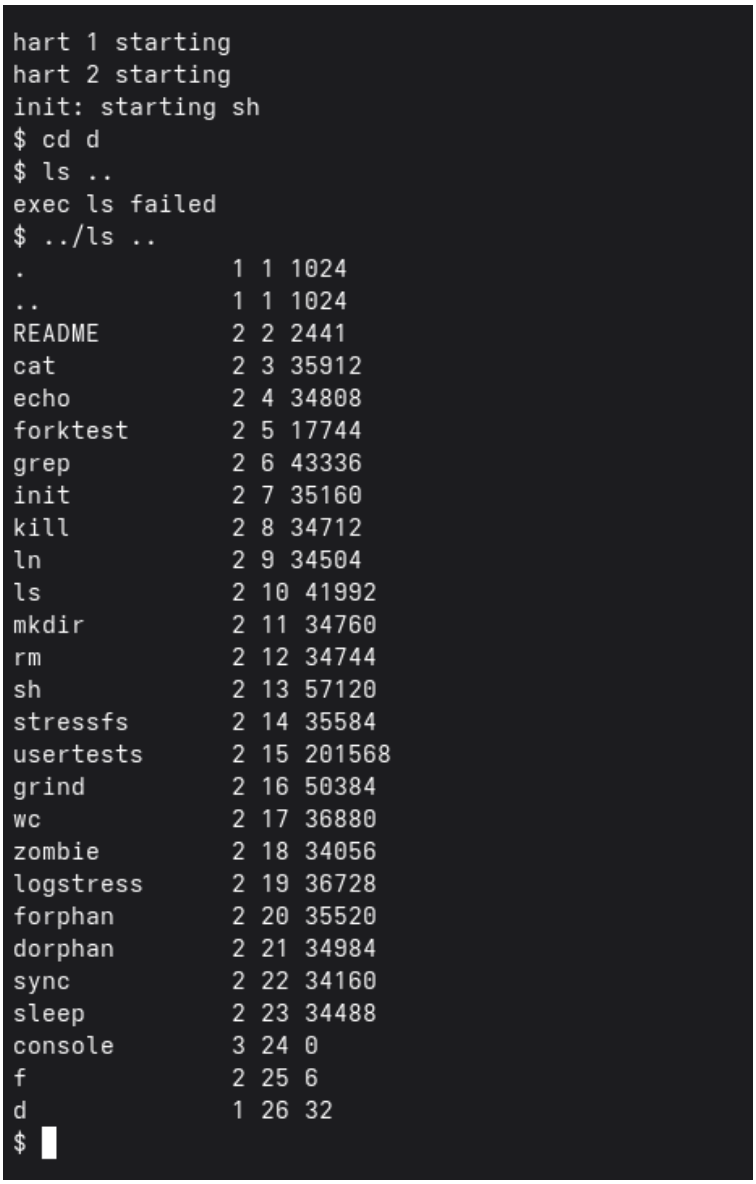


Figure 2.3: Failure of plain ls after cd d and the working ../ls alternative

The missing feature that would be most inconvenient in daily work is command history because repeated long commands would have to be typed again. A minimal `PATH` search would be one of the easiest useful additions: when an executable name contains no slash, the shell could try a short fixed list of directories before reporting failure. Full quote handling and expansion would require deeper tokenizer and parser changes.

3 WHO EXPANDS THE ARGUMENT VECTOR

3.1 Argument-printing programs

The xv6 version includes the xv6 types and user interfaces and terminates with `exit(0)`. The Linux version includes `stdio.h`, uses the same loop, and returns zero from `main`. Both versions print the argument count and every exact string supplied by the shell.

```
printf("argc = %d\n", argc);
for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
```

The xv6 Makefile must contain the tab-indented `$U/_args\` entry. On Linux the source is built with `gcc -o args args_linux.c`.

```
❯ cd ~/xv6-riscv
nano user/args.c
❯ nano Makefile
❯ nl -ba user/args.c
grep -n '\$U/_args' Makefile
     1  #include "kernel/types.h"
     2  #include "user/user.h"
     3
     4  int
     5  main(int argc, char *argv[])
     6  {
     7      printf("argc = %d\n", argc);
     8
     9      for(int i = 0; i < argc; i++)
    10          printf("argv[%d] = [%s]\n", i, argv[i]);
    11
    12      exit(0);
    13  }
154:    $U/_args\
```

Figure 3.1: args.c together with its UPROGS entry

3.2 Observed argument behavior

In a Bash directory containing `args.c` and `args_linux.c`, the command `./args *.c "x y"` produces the following logical argument vector:

```
argc = 4
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [x y]
```

Bash replaces the wildcard with two matching filenames and removes the quote characters while preserving the enclosed space. If the directory contains a different number of `.c` files, `argc` changes accordingly.

In xv6, the corresponding command produces a fundamentally different vector:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

The equal count in this particular example is coincidental. The xv6 shell passes `*.c` literally and treats the space between `x` and `y` as an argument separator because its tokenizer has no quote state. The quote characters remain attached to the two resulting strings.

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ args one              two         three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
$
```

Figure 3.2: Argument vectors printed by args in xv6

```
❯ cd /home/moiseid/University/SO/os-2026-DanielMoisei
cp ~/xv6-riscv/user/args.c lab4/args.c
nano lab4/args_linux.c
'/home/moiseid/xv6-riscv/user/args.c' -> 'lab4/args.c'
❯ cd lab4
gcc -Wall -Wextra -o args args_linux.c
❯ ./args one              two         three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
❯ ./args *.c "x y"
argc = 4
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [x y]
❯ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/moiseid]
❯ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3: Argument vectors printed by args under Bash

These results show that argument expansion is performed by the shell before the program is started. The kernel transfers the already prepared argument array during `exec`, and the program merely receives it. Bash performs pathname expansion and quote removal; xv6 implements neither operation.

The extra spaces in `args one two three` are delimiters. Both shells collapse the unquoted runs for parsing purposes and the program receives the three ordinary arguments `one`, `two`, and `three`, plus `argv[0]`. The original number of spaces cannot be recovered from `argv`.

The remaining captured commands reinforce the same distinction. In xv6, `args $HOME` passes the literal string `$HOME`, while `args 'single quoted'` produces two arguments that still contain the quote characters. Under Bash, `$HOME` expands to `/home/moiseid`, and the single-quoted words arrive as one argument without the quotes.

4 READ THE SHELL SOURCE

4.1 Command representations and dispatch

```
wc -l user/sh.c
grep -n "#define" user/sh.c
grep -n "case " user/sh.c
grep -n "cd" user/sh.c | head
```

In the inspected tree, `wc -l` reported 499 lines. Lines 8 to 12 define the command representations:

```
#define EXEC  1
#define REDIR 2
#define PIPE  3
#define LIST  4
#define BACK  5
```

`EXEC` is produced by an ordinary command with no special operator. `REDIR` is produced by <, >, or two consecutive > characters; `PIPE` by `|`; `LIST` by `;`; and `BACK` by `&`. Parentheses group a nested command tree but do not introduce a sixth command type. The switch in `runcmd` handles the same five values at lines 75, 83, 93, 101, and 125 in the current file.

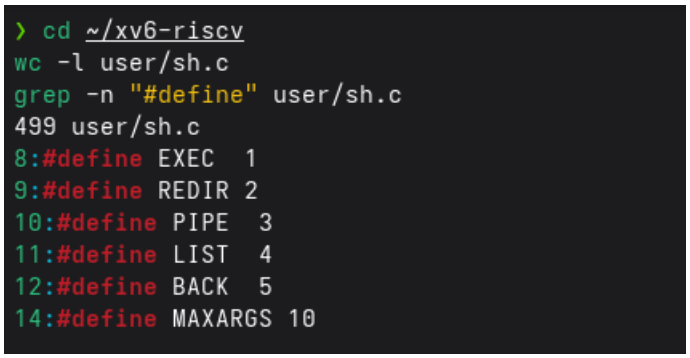


Figure 4.1: The five command kinds defined in user/sh.c

```
❯ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2: The case labels used throughout user/sh.c

4.2 Why cd is special

The main input loop handles `cd` near lines 164 to 172, before forking the process that executes ordinary parsed commands. A directory change affects only the process that calls `chdir`. If `cd` were a separate child program, that child would change its own working directory and then exit, leaving the shell in its original directory. The built-in must therefore run in the parent shell process.

```
❯ grep -n "cd" user/sh.c | head
nl -ba user/sh.c | sed -n '155,180p'
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
   155        break;
   156      }
   157    }
   158
   159    // Read and run input commands.
   160    while (getcmd(buf, sizeof(buf)) >= 0) {
   161      char *cmd = buf;
   162      while (*cmd == ' ' || *cmd == '\t')
   163        cmd++;
   164      if (*cmd == '\n') // is a blank command
   165        continue;
   166      if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
   167        // Chdir must be called by the parent, not the child.
   168        cmd[strlen(cmd) - 1] = 0; // chop \n
   169        if (chdir(cmd + 3) < 0)
   170          fprintf(2, "cannot cd %s\n", cmd + 3);
   171      } else {
   172        if (fork1() == 0)
   173          runcmd(parsecmd(cmd));
   174        wait(0);
   175      }
   176    }
   177    exit(0);
   178  }
   179
   180  void
```

Figure 4.3: The parent-shell code that handles cd

4.3 Connection to the experiments

The tokenizer defines whitespace and only the symbols `<|>&;()`. There is no code that expands `*`, reads `$`-prefixed variables, or interprets quotes. This directly explains why `ls *` receives a literal asterisk, `echo $HOME` prints the variable name, and quoted spaces are not preserved as one xv6 argument. These are shell-language features, not automatic services supplied by the kernel or by the called program.

CONCLUSION

This laboratory showed that even a small shell must parse syntax, build command structures, configure file descriptors, create processes, connect pipes, and preserve parent-process state for built-ins such as `cd`. The updated `sleep` program also demonstrated that a user command can retain its interface while adapting to a renamed system-call wrapper.

The side-by-side experiments separated core process-control features from conveniences implemented by Bash. Both shells can redirect, pipe, sequence, group, and start commands in the background. Bash adds pathname and variable expansion, quote processing, path search, history, completion, and richer job control. The `args` program proved that these transformations occur in the shell: the kernel passes an argument vector, while the receiving program has no knowledge of the original wildcard, quotes, or extra unquoted spaces.

Finally, reading `user/sh.c` connected every visible behavior to a compact implementation. The five command types and the parent-side `cd` special case explain the supported syntax, while the absence of expansion code explains the limitations observed during execution.

REFERENCES

[1] Operating Systems Laboratory Work No. 4. *The xv6 Shell from the Outside*. FCIM / FAF / UTM, 2026-2027.

[2] MIT PDOS. *xv6: a simple, Unix-like teaching operating system*, current RISC-V source tree, files `user/sh.c`, `user/user.h`, and `Makefile`.